

Computer Vision System for Briscola Round Analysis

Final Project Report

Filippo Facco

Francesco Pizzato

September 19, 2026

Abstract

We present a system that watches the video recordings of a two-player Briscola game and rebuilds it round by round: the two cards played, the briscola card, the player who played each card, the leader, the winner and the points of every round, and finally the score of the whole game. Cards are localized by a YOLOv8 oriented-bounding-box detector running in Python, and are then rectified and recognized in C++ with OpenCV by SIFT matching against the 40 reference cards plus a card-back template. Single-frame recognitions are noisy, so the game is not rebuilt frame by frame: we combine all the evidence of the game with frame voting, a global one-to-one assignment solved with the Hungarian algorithm, spatial clustering of the card positions and a persistence criterion for the briscola. On the four provided games we recognize 155/160 played cards, 159/160 player labels and 4/4 briscola cards, and we predict 10/12 final-result fields exactly.

1 Introduction

For each of the four games we receive 20 short clips, one per round, recorded by a fixed camera above the table. The system has to produce a CSV with one row per round,

Round, North_Number, North_Suit, South_Number, South_Suit, Briscola_Number, Briscola_Suit, Leader, Winner, Points
--

and the final totals of the two players with the overall winner.

The main difficulties are well known from the assignment: rotated and partially overlapping cards, hands entering the scene, patterned tablecloths, different lighting conditions, and many irrelevant cards in the frame (the deck, the briscola lying under the deck and the piles of cards already won by the players). Our design choice is to split the problem in two parts:

1. a *perception* part, which answers “where are the cards and which card is each box?” for every sampled frame;
2. a *reasoning* part, which uses the structure of the game (every card of the deck is played exactly once, the briscola never moves, North is above and South is below, the second card is placed next to the first one) to turn these noisy observations into a consistent game.

2 System overview

The program is launched as `./main <game>` and runs four stages (Figure 1):

1. **Detection** (`CardDetector.py`): YOLO [2] finds the cards in the sampled frames and writes their four corners to a JSON file.
2. **Recognition** (`readJSON.cpp`, `CardRectifier.cpp`, `CardRecognizer.cpp`): every detected card is cut from the original frame, rectified and labelled with SIFT [1].
3. **Game reconstruction** (`MatchTheory.cpp`, `FixedMatch.cpp`): briscola, cards of every round, leader, winner and points.

4. **Evaluation** (`Print.cpp`, `PerformanceMeasurement.cpp`): the CSV is written in the ground-truth format and compared with the ground truth.

Labels are integers: 1–10 Spades, 11–20 Clubs, 21–30 Cups, 31–40 Coins, and 0 means “not a card face” (card back or no reliable match).

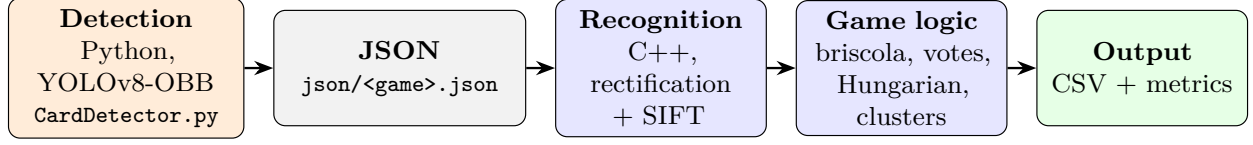


Figure 1: Pipeline of the system. Only the first stage is written in Python; everything else is C++ with OpenCV [3].

3 Detection with YOLO and communication between Python and C++

3.1 Why YOLO is used

Our first card detector used only classical techniques: Gaussian smoothing, Canny edge detection [4], morphological closing and contour analysis, keeping the contours with the area and aspect ratio of a card. On the real videos the results were poor: textured tablecloths, overlapping cards, hands and changes in lighting made the detector miss too many cards to be recovered by the later steps. For this reason we replaced it with a YOLO detector, which gave much better results. The results forced this choice on us. Our goal is to use classical computer vision as much as possible, so we decided to use YOLO only as a *localizer*: it is trained on the single class *card* and says where a card is, not which one. The recognition stays fully classical using SIFT, and the oriented boxes give the four corners needed for the perspective rectification.

3.2 How YOLO is used

Training. The training images were generated synthetically by a Python script written with the help of an AI assistant, using the 40 reference cards and backgrounds taken from the game videos. In this way every image comes with exact oriented labels, without any manual annotation. We then applied transfer learning: starting from the public pretrained `yolov8n-obb.pt` weights [2], we fine-tuned a **YOLOv8n-OBB**. During training we added online augmentations: colour jitter in the HSV space, random translation and scaling, horizontal flips, mosaic and random erasing to simulate occlusions by hands. The result is the file `best.pt`, included in the project folder.



Figure 2: Example of a synthetic image used to train the YOLO detector.

Inference. At inference time (`CardDetector.py`) each round video is opened with OpenCV and we sample one frame every 10 frames, only in the central part of the clip (from 20% to 80% of its length). The beginning and the end of each clip show the table before the play and the collection of the cards: they bring no information about the played cards and contain many hands.

3.3 Python - C++ Communication

The two languages communicate through a **JSON** file and a process call:

1. The C++ `main` checks if `json/<game>.json` exists. If it does not, it launches the detector as an external process with the command `std::system("python3 ../src/CardDetector.py <game>")` and waits for it to finish. If the JSON is still missing afterwards (Python or Ultralytics not installed) the program stops with an explanatory message.

2. The Python script writes a flat list with one entry per detected card:

```
{ "round": 1, "frame": 60, "conf": 0.914,
  "corners": [[427.7,1563.1],[592.2,1503.6],[483.1,1201.7],[318.6,1261.1]] }
```

3. The C++ side parses it with the header-only `nlohmann/json` library [5] (`readJSON.cpp`) into a vector of `JSONCardInfo {round, frame, corners}`, which is then passed to the recognition step.

4 Card recognition

4.1 Rectification

YOLO returns the four corners in an arbitrary order. `orderCorners` sorts them by angle around their centroid (which works for any rotation) and then rotates the sequence so that it starts from the top-left corner (the one with the smallest $x + y$), giving the order TL, TR, BR, BL. The output size is the maximum of the two opposite side lengths, and `cv::getPerspectiveTransform + cv::warpPerspective` produce a straight image of the card, with the same orientation as the reference templates. If the result is landscape (for example the briscola, which lies horizontally under the deck) it is rotated by 90° , so that every card is in portrait orientation. The image is then converted to grayscale.

4.2 SIFT matching

The SIFT keypoints and descriptors of the 41 templates are computed once at start-up. For a detected card we compute its SIFT descriptors and, for every template, we match them with a brute-force matcher (L2 norm, $k = 2$ nearest neighbours) and count the *good matches* that pass the ratio test [1] ($d_1 < 0.75 d_2$). The card receives the label of the template with the most good matches; if the best template has fewer than 15 good matches the card is labelled 0. SIFT is invariant to rotation and scale, so a card that is upside down or at a different distance from the camera is still matched correctly, and the local descriptors are robust to partial occlusion by another card or by a hand.

4.3 Template 0: the card back

The deck, the pile of won cards of each player and the cards covered by the fingers show the back of the card, and YOLO correctly detects them as cards. Without an explicit model of the back, SIFT would always assign them the “least bad” face card, creating many false observations (often more than the real ones) that would pollute the votes of every round. We therefore added a 41st template, `0.jpg`, with the back of a card and label 0. It is inserted *first* in the template list and the best template is updated only with a strictly greater number of matches, so a face card must beat

the back to be accepted; ties go to the back. Every box recognized as the back is simply ignored by the game logic.

5 Game reconstruction

After recognition we have a list of `CardDetection {round, frame, label, center}`, where `center` is the centre of the oriented box. Even after the filtering of the card back, this list still contains errors: some cards are misread in some frames, some are never recognized (hidden by a hand or by the other card), and the list also contains the briscola lying under the deck in every round. All the rules in Table 1 below use many frames and many rounds together instead of trusting a single frame.

Parameter	Meaning
<code>clusterDistance</code>	two boxes closer than this are the same card on the table
<code>briscolaDistance</code>	a box this close to the briscola position is the briscola under the deck
<code>sameColumn</code>	max horizontal distance between the two played cards
<code>minOffset</code>	min vertical distance between the two played cards
<code>frameMiddleY</code>	fallback border between North and South
<code>maxDoubleFrames</code>	tolerated frames where the briscola candidate appears twice

Table 1: Parameters of the game-reconstruction stage.

5.1 Choosing the briscola

The briscola is the same for the whole game. It lies face up under the deck in the same place from the first round until the deck is exhausted: with 40 cards, 6 dealt and 1 briscola, the deck is finished after round 17, so in the last 3 rounds the briscola is in a player’s hand and may even be played. A played card, on the contrary, appears only in the round in which it is played. The briscola is therefore the card that is seen in the largest number of rounds in the same place:

1. for every label ℓ , we compute its usual position as the *median* x and y of all its boxes in rounds $1..N_{rounds} - 3$ (the median ignores the few boxes where the label was assigned by mistake to a card somewhere else);
2. we count in how many different rounds ℓ is recognized within 150px of that position (each round counts once, however many frames);
3. the label with the highest count is the candidate briscola.

Counting rounds in the same place, and not simply detections, is what makes this robust: a card back or a pile misread as some card is seen in different positions and in few rounds, and a card that is played is seen in only one round, while the true briscola is seen in almost every one of the first 17 rounds.

As a further check, a candidate is rejected if it is seen twice in the same frame, in two boxes more than 70px apart, during rounds $1..N_{rounds} - 3$: in those rounds the real briscola is under the deck and cannot be on the table at the same time, so a card that appears twice was actually played and is only being confused with the briscola.

Once the briscola is known, its median position is stored and every box within 150px of it in rounds $1..N_{rounds} - 3$ is removed from the evidence about the played cards (`isBriscolaUnderDeck`). In the last three rounds this filter is disabled, because the briscola can be played like any other card.

5.2 Which two cards were played: frame voting

To choose the cards of every round we started from a standard approach for data association, found in the literature on multi-object tracking [6]: the observations vote in a score matrix, and

the Hungarian algorithm [7, 9] then finds the best one-to-one assignment. We adapted it to our problem as follows. We build a score matrix with $2N_{\text{rounds}} = 40$ rows (*slots*: row $2(r - 1)$ is the North card of round r and row $2(r - 1) + 1$ the South card) and 40 columns (the cards). For every frame of round r we take the recognized cards (label $\neq 0$, not the briscola under the deck; if a label appears twice in the frame only the higher box is kept) and vote:

- two or more cards: the highest card (smallest y) gets +2 in the North slot and the lowest card gets +2 in the South slot;
- a single card: +1 in both slots, because we do not yet know who played it.

A card that is visible in many frames of its round thus collects many votes, while a misreading in one or two frames collects only a few.

5.3 Global assignment with the Hungarian algorithm

The problem. After frame voting every slot has a list of candidate cards, each with a score. The simplest solution would be to pick, for every slot, the card with the highest score. This choice is made independently for each slot, so it ignores the strongest constraint we have about a game of briscola: in a game of 20 rounds exactly 40 cards are played, so every card of the deck is played exactly once. A choice made slot by slot can break this rule in two ways:

- the same card can win in two different slots, because in some frames of another round it was confused with a similar card. We would then predict it twice;
- a card that is never recognized (for example because a hand always covers it) gets no votes, so it is never chosen. Its slot ends up with some wrong card that only has a few votes.

What the Hungarian algorithm does. The rule “one card per slot, and one slot per card” makes the problem an *assignment problem* [7]: a perfect bipartite matching between the 40 slots and the 40 cards. Among all the ways of giving a different card to every slot, we want the one with the largest total score. The Hungarian algorithm (Kuhn–Munkres) [7, 8] solves this problem exactly; our implementation follows the $O(n^3)$ version described in [9]. It does not decide slot by slot. It looks at the whole matrix together, so a slot gives up its best card only when that card is worth more somewhere else, and the total score it loses is as small as possible.

Why we chose it.

- **It uses the rule of the game.** “Every card is played once” is the only constraint that links all the rounds together. The Hungarian algorithm is the natural tool to impose it, and the evidence of one round can correct the mistakes of another.
- **Exact solution.** Trying all assignments is impossible ($40! \approx 8 \cdot 10^{47}$). A greedy method that takes the best pairs one after the other and removes them depends on the order. An early wrong choice cannot be undone and propagates to the other slots. The Hungarian algorithm always returns the global optimum.
- **Fast and simple.** It runs in $O(n^3)$: with $n = 40$ that is a few tens of thousands of operations, so it takes no measurable time compared with YOLO and SIFT. Our implementation (`hungarianMax` in `FixedMatch.cpp`) is about fifty lines and needs no extra library.

Implementation details. The algorithm minimizes a cost, so each score becomes $c = \max(\text{score}) - \text{score}$. Every perfect matching contains exactly n terms, so this transformation does not change which assignment is optimal. The algorithm needs a square matrix. When the video contains fewer than 20 rounds, the matrix is padded with fictitious slots of score 0, and the cards assigned to them are ignored.

The North/South meaning of the two slots of a round is used only to spread the votes. The Hungarian algorithm only decides *which* two cards were played in each round. Which player played each card is decided in the next steps.

5.4 Card positions: clustering

To decide which card was played first and by whom we need the *positions* of the two played cards. For every round we group the boxes by position, without looking at their labels (`buildClusters`): each recognized box (label $\neq 0$, not the briscola under the deck) joins the nearest existing cluster if its centre is closer than 70 px to the cluster's first centre, otherwise it starts a new cluster. Each cluster stores the frames and the labels of its boxes. Ignoring the labels is important: when the second card is placed on top of the first one, the covered card is often read with the label of the card above it, but its position does not change.

The clusters of the two played cards are chosen as follows:

- **first card**: among the clusters seen in at least 3 different frames, the one that appears first in time (`findFirstCluster`). Spurious detections last only one or two frames: a card in a player's hand passing over the table, or the cards being moved when they are collected at the end of the round. The 3-frame threshold removes these short-lived clusters, so the earliest remaining cluster is the card of the leader;
- **second card** (`findSecondCluster`): a cluster seen in at least 2 frames that appears after the first one, lies in the same column ($|\Delta x| < 100$ px) and is shifted up or down ($|\Delta y| > 60$ px), as the players place their card above or below the opponent's one. If several clusters qualify we prefer the one that contains one of the two cards chosen by the Hungarian algorithm, and then the one seen in more frames.

5.5 Leader, players, winner and points

[da rivedere]

Let c_1, c_2 be the two cards assigned to round r .

1. **Order.** For each card we find the first frame in which it is recognized (ignoring the briscola under the deck). If both are found in different frames, the earlier card is the leader's card. Otherwise (one card never recognized, or both appear together) the order is taken from the second cluster: the card that is voted most in that cluster is the second card.
2. **Players.** If both clusters exist, the one with the smaller y (higher in the image) is North's card and the other is South's. If only one cluster exists, it is assigned to the player on its half of the table. The border between the halves is estimated from the data as the average of the midpoints between the two clusters in all the rounds where both are found (falling back to half of the frame height). This makes the rule independent from where exactly the camera frames the table.
3. **Winner.** If exactly one card has the briscola suit, it wins; if the suits are different and none is a briscola, the leader wins; if the suits are equal, the stronger card wins, with the order $1 > 3 > 10 > 9 > 8 > 7 > 6 > 5 > 4 > 2$.
4. **Points.** The winner takes the sum of the points of the two cards ($1 \rightarrow 11, 3 \rightarrow 10, 10 \rightarrow 4, 9 \rightarrow 3, 8 \rightarrow 2$, others 0). The final score is the sum over the rounds, and the game winner is the player with more than 60 points (a draw at 60–60).

6 Evaluation

The predicted rows are written to `output/<game>_output.csv` in the exact ground-truth format and read back, so the rows printed on screen, the CSV and the rows used for the metrics are exactly the same. `PerformanceMeasurement.cpp` normalizes the text fields (trimming and lower case) and computes the four metrics of the assignment: card recognition (40, rank and suit must both match), player identification (40, leader and winner of every round), briscola recognition (1) and game result (3, winner and the two final scores). Every wrong field is also listed in `output/<game>_metrics.txt`. The comparison is made against the corrected ground-truth files `<game>resultsCORRECTED.csv`.

6.1 Results

Game	Cards	Players	Briscola	Result
game1 (controlled)	40/40	40/40	1/1	3/3
game2 (different lighting)	37/40	40/40	1/1	3/3
game3 (realistic play)	38/40	39/40	1/1	1/3
game4 (different background)	40/40	40/40	1/1	3/3
Total	155/160 (96.9%)	159/160 (99.4%)	4/4	10/12

Table 2: Metrics on the four games.

Game	Predicted (North – South, winner)	Ground truth
game1	53 – 67, South	53 – 67, South
game2	59 – 61, South	59 – 61, South
game3	74 – 46, North	70 – 50, North
game4	63 – 57, North	63 – 57, North

Table 3: Final scores.

Games 1 and 4 are reconstructed without any error, including game 4 where the different background produces many more detections (1250 against 355–591 in the other games). The briscola is correct in all four games, so the persistence criterion of Section 5.1 never failed.

6.2 Error analysis

Game 2. Three card fields are wrong. In round 1 North’s 7 of Coins is read as the 6 of Coins, two cards with a very similar layout of pips. Because of the global assignment, the 7 of Coins then remains available and is given to round 12, where it takes North’s slot, and the 7 of Cups moves to South’s slot in place of the 6 of Coins. This shows the other side of the Hungarian algorithm: it corrects isolated errors, but when a card is systematically confused with another one the error can move to a second round. All the cards involved are worth 0 points and leader and winner are unchanged, so the players metric and the final score are still perfect.

Game 3. This is the realistic game, with hands over the cards, overlaps and no clean pause before the collection. In round 4 the two cards (6 and 10 of Coins) are recognized correctly but are assigned to the wrong players: the leader (North) is correct, but the system considers the 10 of Coins as the first card played, while North actually played the 6 and South answered with the 10. The 6 was probably recognized only after the 10, because it was covered while it was placed on the table, so the first-recognition order was reversed. As a consequence the winner of the round is wrong and its 4 points go to North instead of South: the final score becomes 74–46 instead of 70–50, although the winner of the game (North) is still correct. A single swap in one round is enough to lose two of the three result fields.

7 Group contributions

The overall pipeline was designed together. The first step was to decide how to split the work between a learned detector and classical methods. We chose YOLOv8-OB in Python only to *localize* the cards, and SIFT matching in C++ to *recognize* them. Once the single-frame recognition proved not to be reliable enough, we moved most of the effort to the game-level reasoning: the card-back template, frame voting with the Hungarian algorithm, and the label-free position clusters.

This produced a large improvement in the reconstructed games. Both members took part in testing and validation on all four games of the dataset.

Area	Filippo Facco	Francesco Pizzato
Ideas & Design	Joint pipeline discussion; proposed the card-back template (label 0) to discard cards not in play, and frame voting followed by a global assignment with the Hungarian algorithm	Joint pipeline discussion; proposed choosing the briscola as the card seen in the most rounds in the same place, and label-free position clusters to find the leader
Implementation	<code>CardRecognizer.cpp</code> , <code>FixedMatch.cpp</code> , <code>Print.cpp</code> , <code>PerformanceMeasurement.cpp</code>	<code>readJSON.cpp</code> , <code>CardRectifier.cpp</code> , <code>MatchTheory.cpp</code> , <code>main.cpp</code> , <code>CardDetector.py</code>
Testing	Inspected the recognized labels frame by frame; tuned the SIFT thresholds	Ran and debugged the pipeline on all 4 games; tuned the distance parameters of Table 1
Performance Measurement	Implemented the comparison with the ground truth (per-round accuracy of cards, players, winner and points)	Checked and corrected the ground-truth files; cross-checked the metrics
Working Hours	~100 hours	~110 hours

Table 4: Group contributions breakdown per member.

8 Conclusion

We are satisfied with the result: the system rebuilds three of the four games perfectly in terms of score and winner, and finds the correct winner in the realistic game. This project was also an important experience for us. It taught us how to organize and manage a large project with many files and modules, and how to make two languages, Python and C++, work together in a single pipeline. It gave us a real-word practical experience with deep learning, from building a training set to fine-tuning a detector, and it showed us how to use classical computer vision on real, noisy videos instead of clean examples. Finally, it taught us that single-frame results do not need to be perfect when the knowledge of the problem is used to correct them.

References

- [1] D. G. Lowe, Distinctive image features from scale-invariant keypoints, *International Journal of Computer Vision*, 60(2):91–110, 2004.
- [2] G. Jocher, A. Chaurasia, J. Qiu, Ultralytics YOLOv8, 2023. <https://github.com/ultralytics/ultralytics>.
- [3] G. Bradski, The OpenCV Library, *Dr. Dobb’s Journal of Software Tools*, 2000.
- [4] J. Canny, A computational approach to edge detection, *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 8(6):679–698, 1986.
- [5] N. Lohmann, JSON for Modern C++. <https://github.com/nlohmann/json>.
- [6] A. Bewley, Z. Ge, L. Ott, F. Ramos, B. Upcroft, Simple Online and Realtime Tracking, in *Proc. IEEE ICIP*, pp. 3464–3468, 2016.
- [7] H. W. Kuhn, The Hungarian method for the assignment problem, *Naval Research Logistics Quarterly*, 2(1–2):83–97, 1955.
- [8] J. Munkres, Algorithms for the assignment and transportation problems, *Journal of the Society for Industrial and Applied Mathematics*, 5(1):32–38, 1957.
- [9] cp-algorithms, Hungarian algorithm for solving the assignment problem, <https://cp-algorithms.com/graph/hungarian-algorithm.html>.